

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 07

NOMBRE COMPLETO: José Eduardo Martínez García

Nº de Cuenta: 320304679

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 14/10/2026

CALIFICACIÓN: _____

Reporte de práctica 5: Iluminación 1

1. **Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.**

a) Añadir una luz de faro (spotlight) frontal al rover con jerarquía **suyo** que ilumine hacia adelante y al avanzar o retroceder el Rover, la luz siga unida a él.

Primero que nada, para poder lograr este requerimiento se sustituyeron los modelos previos del rover, el cubo y el helicóptero por los del avión con texturizado de cars, el rover con jerarquía en código, el cubo y el holocron con texturizado de logos del starwars.

Estos modelos se habían hecho antes en otras prácticas y ejercicios de clase. Después se procedieron a importar dentro de la escena de Open GL.

Después se modificaron y crearon luces tipo spot para ponerlas como faros frontales del rover y el avión, además de la luz puntual de la parte de debajo del avión texturizado con temática del universo de cars.

Definición de las luces spotlights:

```
		//>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
		//>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
		//>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>

		//luz direccional, sólo 1 y siempre debe de existir

		//>>>>>>>>>>>>>>>>>>>>>>>>> LUZ DEL SOL <<<<<<<<<<<<<<<<<<<<<<<<<
		mainLight = DirectionalLight(1.0f, 1.0f, 1.0f,
					0.3f, 0.6f,
					1.0f, -1.0f, -1.0f);
		//contador de luces puntuales
		unsigned int pointLightCount = 0;

		//>>>>>>>>>>>>>>>>>>>>>>>>> LUZ PUNTUAL 1 (LUZ ROJA) <<<<<<<<<<<<<<<<<<<<<<<<<
		pointLights[0] = PointLight(1.0f, 0.0f, 0.0f,
					0.0f, 1.0f,
					-6.0f, 1.5f, 1.5f,
					0.3f, 0.2f, 0.1f); //no pueen ser los tres 0 (0f,0f,0f) debido a que puede causar indeterminacion
		pointLightCount++;

		//>>>>>>>>>>>>>>>>>>>> LUZ SPOT 1 (LINTERNA DE LA CAMARA) <<<<<<<<<<<<<<<<<<<<<
		unsigned int spotLightCount = 0;
		spotLights[0] = SpotLight(0.5f, 0.5f, 0.5f,
```

```
            0.0f, 2.0f,
            0.0f, 0.0f, 0.0f,
            -1.0f, 0.0f, 0.0f,
            1.0f, 0.0f, 0.0f,
            5.0f);
    spotLightCount++;

    //>>>>>>>>>>>>>>> LUZ SPOT 2 (FARO DE FRONTAL DEL AVION) <<<<<<<<<<<<<<<<<
    spotLights[1] = SpotLight(0.0f, 1.0f, 0.0f,
            0.0f, 4.0f,
            0.0f, 0.0f, 0.0f,
            0.0f, 0.0f, 1.0f,
            1.0f, 0.0f, 0.0f,
            45.0f);
    spotLightCount++;

    //>>>>>>>>>>>>>>> LUZ SPOT 3 (FARO DEL AVION BAJO) <<<<<<<<<<<<<<<<<<
    spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f,
            0.0f, 3.0f,
            0.0f, 0.0f, 0.0f,
            0.0f, -1.0f, 0.0f,
            1.0f, 0.0f, 0.0f,
            25.0f);
    spotLightCount++;

    //>>>>>>>>>>>>>>> LUZ SPOT 3 (FARO DE FRONTAL DEL ROVER) <<<<<<<<<<<<<<<<
    spotLights[3] = SpotLight(1.0f, 0.0f, 1.0f,
            0.0f, 5.0f,
            0.0f, 0.0f, 0.0f,
            0.0f, 0.0f, 1.0f,
            1.0f, 0.0f, 0.0f,
            45.0f);
    spotLightCount++;
```

Después se colocó la luz por medio de jerarquía unida al centro del rover para que se mueva con él. Guarda la transformación sin escala. Es la que heredan las luces y cualquier pieza hija

```
            // >>>>>>>>>> Jerarquía del rover <<<<<<<<<<<
            glm::vec3 roverPos = roverBase + ROVER_AVANCE * mainWindow.getmuevex();

            glm::mat4 modelRoverAux = glm::mat4(1.0f);
            modelRoverAux = glm::translate(modelRoverAux, roverPos);
            modelRoverAux = glm::rotate(modelRoverAux, glm::radians(180.0f),
                                        glm::vec3(0.0f,1.0f,0.0f));
```

Una luz necesita posición y dirección en el mundo, pero se coloca la ubicación donde en este caso va a estar en la trompa o punta del rover, esto para no quedar al centro de la figura. faro

al frente (offset (4, 1, 0)) apuntando hacia muevex. Como las piezas hijas ya usan modelaux, por lo que se mueven con el cuerpo y con el faro sin tocar nada más.

```
// Faro delantero
glm::vec3 posFaroRover = glm::vec3(modelRoverAux * glm::vec4(4.0f, 1.0f, 0.0f, 1.0f));
glm::vec3 dirFaroRover = glm::normalize(glm::vec3(modelRoverAux * glm::vec4(ROVER_AVANCE, 0.0f)));
spotLights[1].SetFlash(posFaroRover, dirFaroRover);
```

Para las entradas de teclado se definió la variable muevex dentro de Window.cpp

```
muevex = 2.0f;
```

Además de la condición de la entrada:

```
if (key == GLFW_KEY_Y)
{
	theWindow-> muevex += 1.0;
}

if (key == GLFW_KEY_U)
{
	theWindow-> muevex -= 1.0;
}
```

Salida de la implementación del ejercicio 1 en la ventana de la escena de OpenGL:

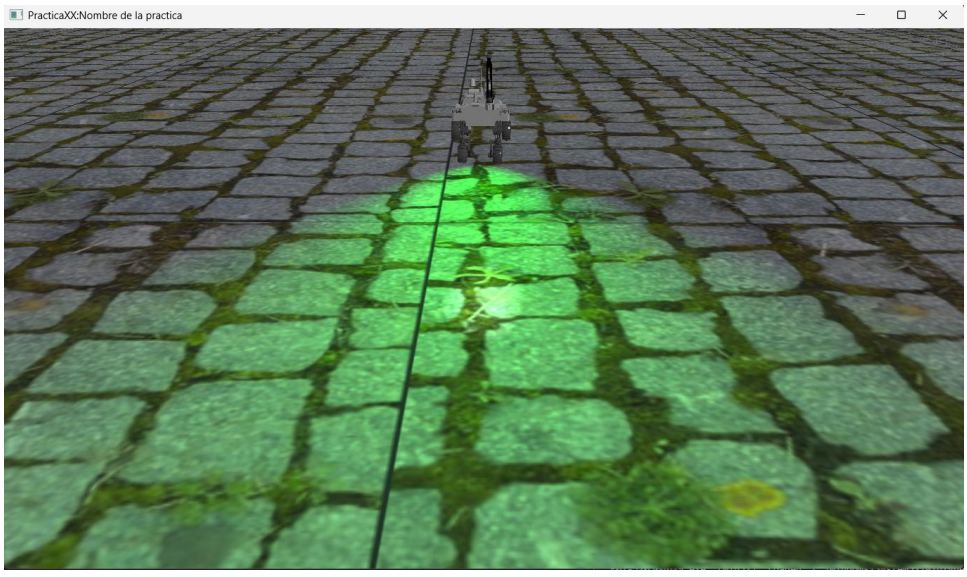


Imagen 1. Rover con iluminación con faro frontal vista frontal



Imagen 2. Rover con iluminación con faro frontal vista trasera superior



Imagen 3. Rover con iluminación con faro frontal vista lateral

Las teclas de entrada de movimiento del avión:

1. Movimiento en +X (frente)
   Tecla Y
2. Movimiento en -X (atrás)
   Tecla U

b) Añadir una luz de faro (spotlight) al avión con jerarquía **suyo** que ilumine hacia abajo y al subir y bajar el avión, la luz siga unida a él.

Se realizaron las mismas acciones que con el rover pero en este caso en la punta del avión texturizado con estilo del universo de cars. Tanto la luz frontal del faro como la de debajo del avión eran spotlights las cuales se tenían que mapear en los respectivos lugares y direccionar hacia los lugares indicados. Para la llamada de las luces definidas fuera del while y para la llamada de las entradas de teclado se utilizó esta sección de codigo:

```
// >>>>>>>>> Jerarquía del avión <<<<<<<<<<
GLfloat altura = mainWindow.getavionAltura();

glm::vec3 avionPos = avionBase
        + AVANCE * mainWindow.getavionAvance()
        + glm::vec3(0.0f, altura, 0.0f);


glm::mat4 modelAvion = glm::mat4(1.0f);
modelAvion = glm::translate(modelAvion, avionPos);
glm::mat4 modelAvionAux = modelAvion;
modelAvion = glm::scale(modelAvion, glm::vec3(1.5f, 1.5f, 1.5f));

// Faro 1: bajo
glm::vec3 posAbajo = glm::vec3(modelAvionAux * glm::vec4(0.0f, -0.3f, 0.0f, 1.0f));
glm::vec3 dirAbajo = glm::normalize(glm::vec3(modelAvionAux * glm::vec4(0.0f, -1.0f,
0.0f, 0.0f)));
spotLights[2].SetFlash(posAbajo, dirAbajo);

// Faro 2: delantero
glm::vec3 AVANCE = glm::vec3(-1.0f, 0.0f, 0.0f);
glm::vec3 posFrente = glm::vec3(modelAvionAux * glm::vec4(AVANCE * 1.0f, 1.0f));
glm::vec3 dirFrente = glm::normalize(glm::vec3(modelAvionAux *
glm::vec4(AVANCE, 0.0f)));
spotLights[3].SetFlash(posFrente, dirFrente);
```

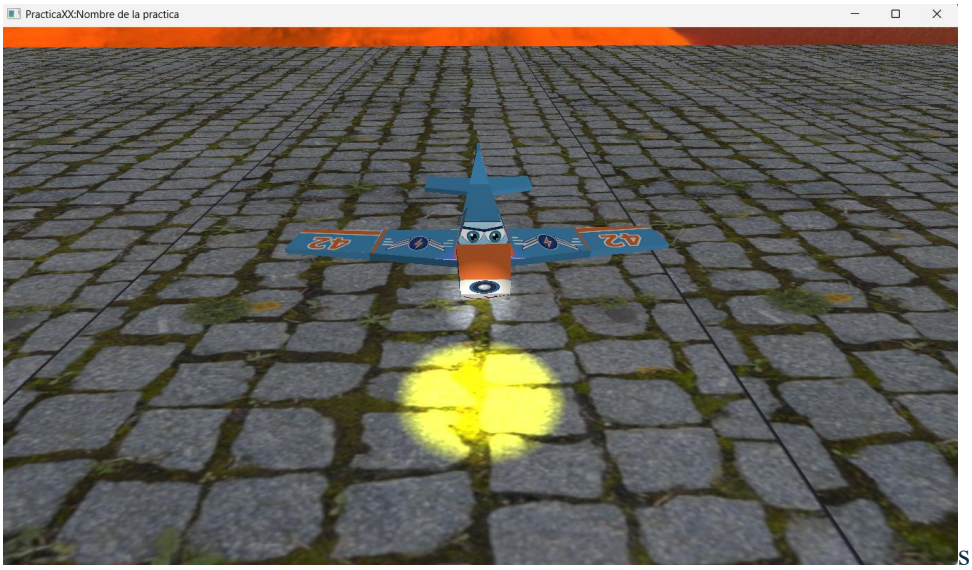
Para las entradas de teclado se definió la variable muevex dentro de Window.cpp

```
avionAvance = 0.0f;
avionAltura = 0.0f;
```

Además de la condición de la entrada:

```
if (action == GLFW_PRESS || action == GLFW_REPEAT)
{
        if (key == GLFW_KEY_F)    theWindow->avionAvance += 0.5f;
        if (key == GLFW_KEY_G)  theWindow->avionAvance -= 0.5f;
        if (key == GLFW_KEY_H)     theWindow->avionAltura += 0.5f;
        if (key == GLFW_KEY_J)     theWindow->avionAltura -= 0.5f;
}
```

Salida de la implementación del ejercicio 2 en la ventana de la escena de OpenGL:



s

Imagen 4. Avion con iluminación con faro inferior vista frontal

c) Añadir una luz de faro (spotlight) al avión con jerarquía **suyo** que ilumine hacia adelante y al avanzar o retroceder el avión, la luz siga unida a él.

Después de la definición de las luces fuera del while y la llamada junto con los modelos de modo de que por jerarquía quedaran unidos a estos al momento del movimiento:

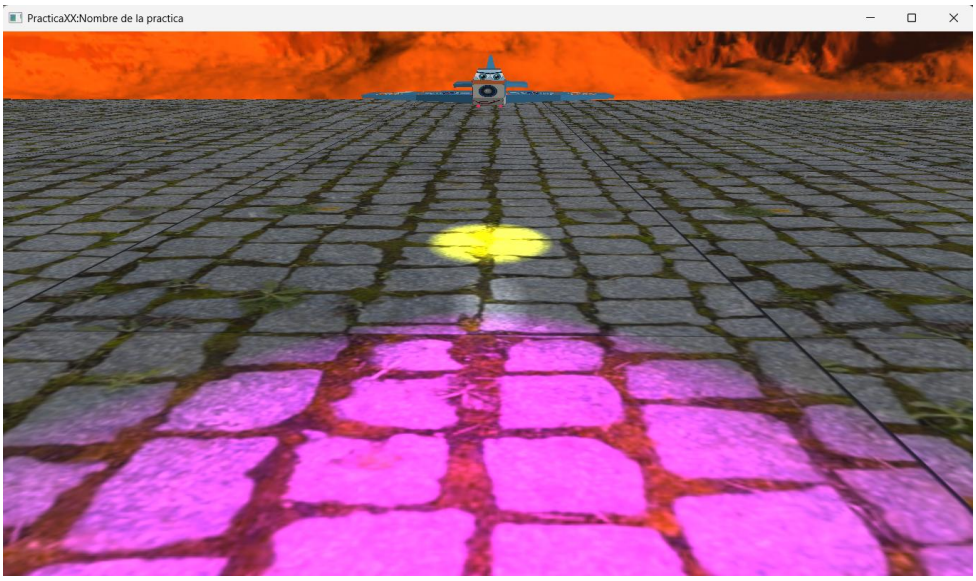


Imagen 5. Avion con iluminación con faro frontal e inferior vista frontal

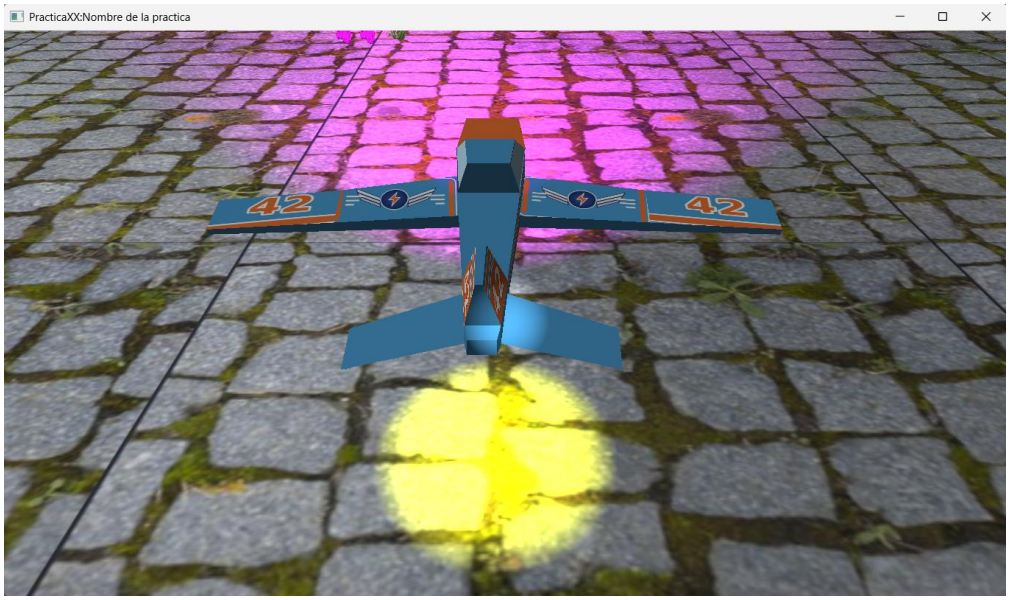


Imagen 5. Avion con iluminación con faro frontal e inferior vista trasera superior

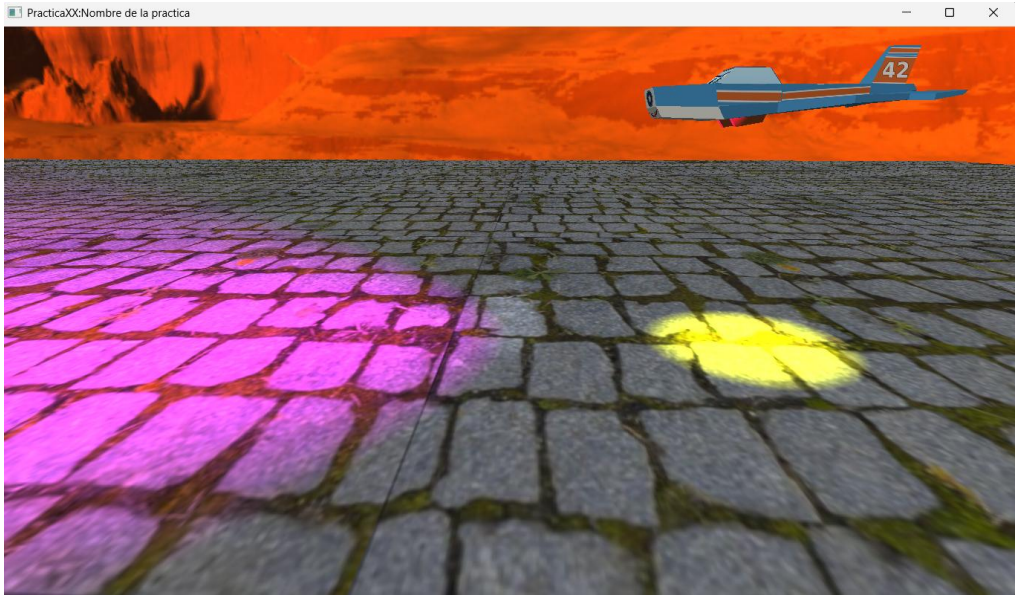


Imagen 5. Avion con iluminación con faro frontal e inferior vista lateral

Las teclas de entrada de movimiento del avion:

3. Movimiento en +X (frente)
   Tecla F
4. Movimiento en -X (atrás)
   Tecla G
5. Movimiento en +Y (arriba)
   Tecla H
6. Movimiento en -Y (abajo)
   Tecla J

2.Problemas presentados a la hora de resolver los ejercicios:

- Al principio no veía nada de iluminación, aunque las texturas sí se veían. Resulta que en el fragment shader la línea que multiplica la textura por la luz estaba comentada, así que las luces se calculaban pero nunca se aplicaban. Además, mis cambios al shader no se reflejaban porque estaba editando una copia distinta a la que cargaba el proyecto. Lo solucioné revisando cuál carpeta shaders leía realmente el ejecutable.
- En el dado, la cara frontal se iluminaba más que las demás sin que la linterna apuntara hacia ella. Me di cuenta de que el shader necesita las normales apuntando al lado contrario de la luz, y que mi luz direccional con ambiente en 0 solo le pegaba a esa cara. Corregí la normal de esa cara y probé una dirección diagonal con algo de luz ambiental para ver que todas se iluminaran distinto.
- El modelo del avión de Blender salía completamente negro. Pensé que era la ruta de la textura en el .mtl, pero cambiarla no sirvió. Revisando Model.cpp encontré dos errores: una condición mal escrita en RenderModel que nunca activaba la textura con un solo material, y un procesamiento de ruta que solo buscaba \ cuando Blender exporta con /. Al corregir ambos, la textura se vio bien.
- Con los faros del avión, al moverlo con el teclado se movía la luz pero el modelo se quedaba quieto. El avión seguía dibujándose con su translate fijo, mientras las luces usaban otra matriz. Lo arreglé usando la misma matriz (modelAvionAux) para las luces y para dibujarlo, igual que con modelaux. También tenía AVANCE declarada dos veces con signos opuestos, y dejé solo una.
- Me confundió por qué la spotlight verde estaba encendida si aún no programaba luz para el rover. Era la luz fija de la práctica original, que se manda al shader en cada frame por el spotLightCount. La reutilicé como faro del rover, calculando su posición a partir de la matriz del cuerpo.

3.Conclusión:

a) Lo más importante fue entender que para que un faro siga a un objeto, la luz y el modelo deben salir de la misma matriz, y que esa matriz se calcula antes de mandar las luces al shader. La posición de la luz se obtiene multiplicando su posición local por la matriz del objeto, y la dirección igual, pero sin que se traslade.

b) Lo más difícil fue encontrar por qué no se veía la iluminación y por qué el avión salía negro, porque no había errores de compilación. Aprendí a descartar causas una por una, por ejemplo, mostrando solo la textura o solo las normales en el shader.

c) Se cumplieron los ejercicios: la escena con las luces, el dado y el holocron con sus normales, el avión texturizado con un faro hacia abajo y otro hacia adelante que lo siguen al moverse, y el rover importado por partes con su propio faro. Para el movimiento reutilicé la lógica de las articulaciones, pero acumulando un desplazamiento en lugar de un ángulo.